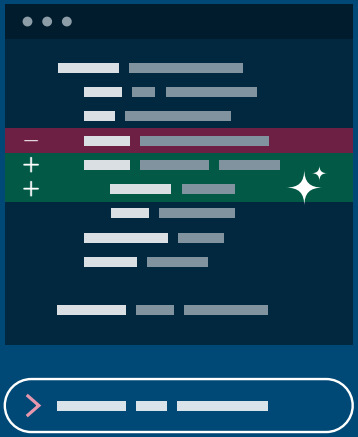
UNIVERSITÄT
DES
SAARLANDES

Coding Agents 101

Workflows & Best Practices



Camilo Martínez

d3M – Data-Driven Design of Materials
Chair of Experimental Methods in Materials Science
Universität des Saarlandes
Research group talk, October 2026

Outline

1	**From asking questions to delegating work**	What must we provide, and what must we verify?
2	**Environments and code checks**	Where does our code run, and what checks it?
3	**Preparing the repository**	What can the repository tell the agent?
4	**Architecture Decision Records (ADRs)**	Why does the project work this way?
5	**Version control with git**	How do we keep and review every change?
6	**Tests first**	How do tests keep an agent on track?
7	**Skills and an end-to-end workflow**	How does a request become a reviewed change?
8	**A realistic starting point**	What should I change in my repository first?



01

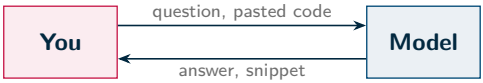
From asking questions to delegating work

What must we provide, and what must we verify, before an agent takes over a task?

From a chat assistant to a coding agent

Conversational assistant

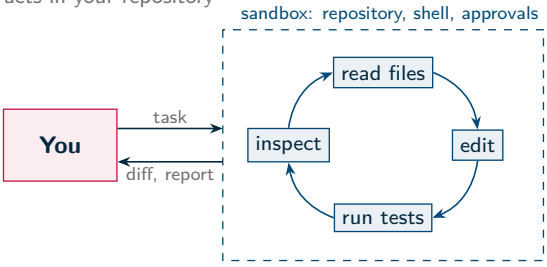
answers questions about what you paste



you run the code; it sees only what you paste

Coding agent

acts in your repository



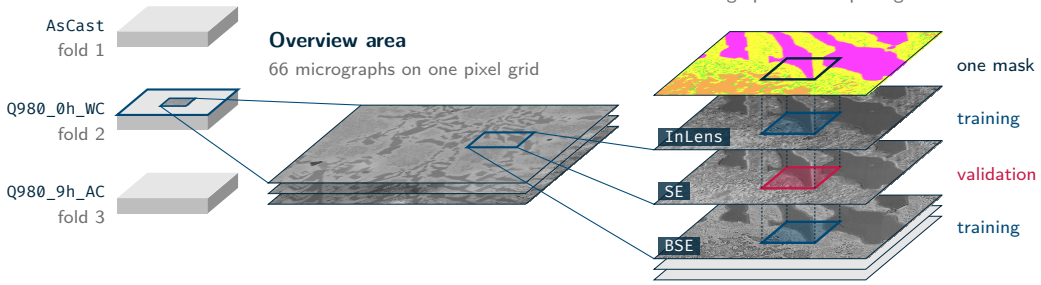
loops until done; you set limits, accept or reject

Running example: split the data by specimen

Specimens one per state

Detail area inside it, 5× finer

60 micrographs on one pixel grid



Random patch split one area, one mask on both sides: data leakage

Task validation folds by specimen, every specimen in exactly one fold

A weak request

the request

> Update the dataset code so we split our SEM data by sample instead of randomly, and add a test.

Objective	Context	Constraints	Acceptance	Verification
? which module, function and CLI option? one split or several folds?	? "sample": a specimen, an area or a micrograph?	? may it rewrite `data/`? add dependencies? *not in the request*	? what must the test show? a length check passes despite leakage	? which command shows that it works? what to report? *not in the request*

what the agent must answer, or guess

A bounded task specification

Objective	Context	Constraints	Acceptance	Verification
add `specimen_folds()` to `src/sem_seg/data/splitting.py`; use it in the training CLI as `--split specimen`	"sample" means specimen: one physical sample, column `specimen_id` of the manifest	leave `data/raw/` and the manifests unchanged; no new dependencies; no GPU jobs	each specimen in exactly one fold; each patch inherits its specimen's fold; same seed, same folds; `ValueError` if `specimen_id` is missing or there are fewer specimens than folds	`pytest tests/data -q` passes; report changed files, test output and assumptions

A bounded task specification

Objective	Context	Constraints	Acceptance	Verification
add `specimen_folds()` to `src/sem_seg/data/splitting.py`; use it in the training CLI as `--split specimen`	"sample" means specimen: one physical sample, column `specimen_id` of the manifest	leave `data/raw/` and the manifests unchanged; no new dependencies; no GPU jobs	each specimen in exactly one fold; each patch inherits its specimen's fold; same seed, same folds; `ValueError` if `specimen_id` is missing or there are fewer specimens than folds	`pytest tests/data -q` passes; report changed files, test output and assumptions

But ...

- too much to write for every task
- what if the agent knows a better way?

02

Environments and code checks

Where does the agent run our code,
and what checks it automatically?

The classic setup: a conda environment

environment.yml

```yaml
name: sem-seg
channels:
  - conda-forge
dependencies:
  - python=3.12
  - numpy
  - pandas
  - pytorch      # with CUDA builds
  - pytest
```

Python-only projects: uv is faster

terminal

```
$ conda env create -f environment.yml
$ conda activate sem-seg
$ conda run -n sem-seg pytest -q    # no activation
```

an agent session: a new shell for every command

```
$ conda activate sem-seg
```
sem-seg

```
$ pytest -q
```
not sem-seg

```
$ conda run -n sem-seg pytest -q
```
sem-seg

or: activate first, then start the agent

Ruff, mypy and Pylance

Ruff

how the code is written

linter and formatter

```
import os  unused import

x=[1,2 ,3] → x = [1, 2, 3]
```

runs in VS Code terminal

mypy

whether the types match

type checker: reads the type hints

```
n_folds: int = 3  ✓

n_folds: int = "3"  not an int
```

runs in VS Code terminal

Pylance

help while you type

VS Code's language server

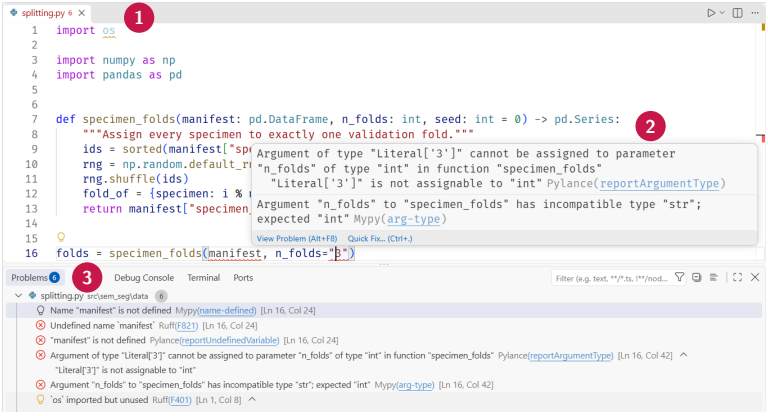
```
specimen_folds(
  (manifest, n_folds: int)
```

and type checks, once switched on

runs in VS Code

together: Pylance while you write; Ruff and mypy as the checks you and the agent run

What it looks like in VS Code



1. Ruff: unused import (`F401`), undefined name (`F821`)
2. Pylance and mypy: a string passed as `n_folds`
3. Problems panel: every finding, with its tool and rule

Set them up once

pyproject.toml

```toml
[tool.ruff]  # rules: Ruff's defaults
line-length = 100

[tool.mypy]
python_version = "3.12"
```

Install	`conda install -c conda-forge ruff mypy`
Lint	`ruff check --fix`
Format	`ruff format`
Types	`mypy src`

.vscode/settings.json

```jsonc
{
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.formatOnSave": true,
    "editor.codeActionsOnSave": {
      "source.fixAll": "explicit",
      "source.organizeImports": "explicit"}},
  "python.analysis.typeCheckingMode": "standard",
  "mypy-type-checker.importStrategy": "fromEnvironment"
}
```

- VS Code extensions: Python (brings Pylance), Ruff, Mypy Type Checker
- pick the environment: *Python: Select Interpreter*
- the same commands go into `AGENTS.md`

03

Preparing the repository

What can the repository tell the agent, so that we do not have to?

Setting up the repository

A standard layout

cookiecutter-data-science

```
sem-seg/
├── data/raw/              never edited
├── data/processed/        made by the code
├── models/                trained models
├── notebooks/             exploring
├── reports/figures/       figures for papers
├── src/sem_seg/           the code
├── tests/
├── AGENTS.md              for the agent
├── environment.yml
├── pyproject.toml
└── README.md              for people
```

As the project grows

```
sem-seg/
├── data/ …                as before
├── models/ … reports/
├── src/sem_seg/, tests/
├── AGENTS.md              commands, rules, pointers
├── GLOSSARY.md            the project's words
├── CODING_STANDARDS.md    how code is written here
└── environment.yml, …
```

AGENTS.md keeps what every session needs and points to the two new files

AGENTS.md

AGENTS.md

```
# sem-seg
## Setup and checks
- Environment: conda env `sem-seg`, activated beforehand
- Fast check (CPU, synthetic data): `pytest -q`
- Lint, types: `ruff check`, `mypy src` ← after every edit
## Conventions
- Terms: `GLOSSARY.md` (specimen, micrograph, patch) 🔗
- Decisions: `docs/adr/` 🔗
- Lengths in micrometres; pixel size from metadata
## Boundaries
- Never write to `data/raw/`; use `data/processed/`
- No cluster jobs (`sbatch`): the queue is shared
- Ask before adding a dependency
## When you finish
- Report changed files, checks run, open assumptions
```

Put in

- ✔ commands to set up, test and check
- ✔ pointers to what the code cannot show
- ✔ what needs your approval
- ✔ when it is done, and what to report
- ✔ the reason behind a rule

Leave out

- ✘ rules that clash or are out of date
  the agent may stop and wait
- ✘ vague wishes: “write clean code”
  name the exact mistake instead
- ✘ “think carefully”, “be thorough”
  current models already do

GLOSSARY.md and CODING_STANDARDS.md

holds term and meaning | words to avoid | code name

GLOSSARY.md read when pointed to

```
# Glossary
Specimen: one physical sample (`specimen_id`).
  Avoid: sample, coupon. ← the word in our request
Area: one imaged region of a specimen.
Micrograph: one SEM image of an area
  (`micrograph_id`). Avoid: image, frame.
Patch: a crop of a micrograph used in
  training (`patch_id`). Avoid: tile.
Fold: specimens held out together; every
  specimen belongs to exactly one fold.
```

holds seeds | input checks | tests | paths

CODING_STANDARDS.md read in review

```
# Coding standards
- Every random step takes a `seed`; the
  same seed gives the same result.
- Public functions check their inputs and
  raise `ValueError` naming the problem.
- Tests call public functions on small,
  hand-written fixtures, never real data.
- Expected values come from the fixture,
  never recomputed the way the code does.
- Paths come from `config.py`, never
  written into a function.
```

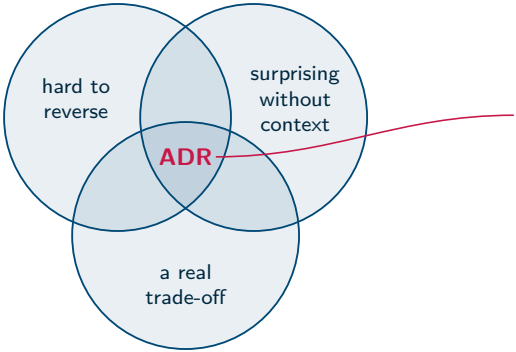
04

Architecture Decision Records (ADRs)

Why does the project work this way,
and what if we change our minds?

What an ADR is, and when to write one

One decision per record: title · context · decision · status · consequences; numbered; superseded records stay



typical in research code

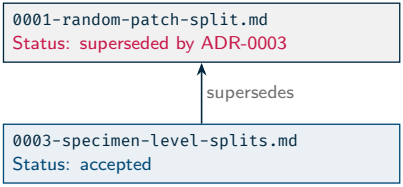
- the split unit and the evaluation protocol
- merged or remapped classes
- units, scales, normalisation
- the metric that decides what is “better”

no ADR: plot styling, renames

ADR-0003: specimen-level splits

docs/adr/0003-specimen-level-splits.md

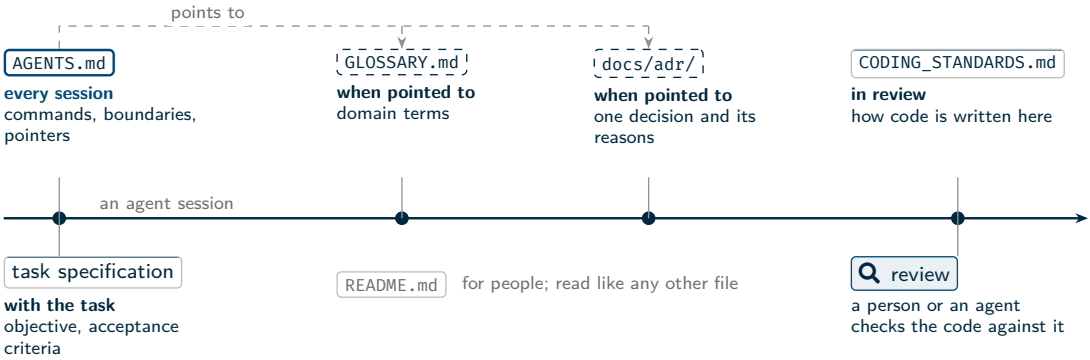
```
# ADR-0003: Specimen-level splits
Status: accepted. Supersedes ADR-0001 (random patch split).
## Context
Micrographs of one specimen share preparation and etching;
several show the same area. A random patch split puts this
material in training and validation.
## Decision
Specimens are assigned to validation folds; each patch
inherits its specimen's fold. Folds are saved with seed
and manifest hash and checked for overlap.
## Considered options
Random patch split; split by micrograph. Both leak.
## Consequences
Few independent units: report every fold and its spread.
One specimen per state: a held-out specimen is a new state.
```



ADR-0001 stays: it shows what was tried and why it changed.

> An agent chasing a better score finds ADR-0003 through `AGENTS.md` before it brings back a random split.

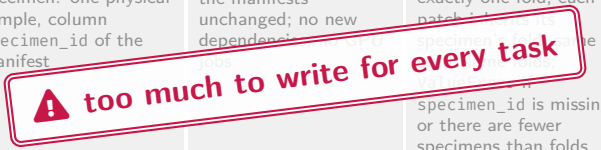
When the agent reads each file



OpenAI (2026), Custom instructions with AGENTS.md ·
Pocock (2026), mattpocock/skills v1.3.1: code-review; v1.3 changelog

Remember the bounded task specification?

Objective	Context	Constraints	Acceptance	Verification
add `specimen_folds()` to `src/sem_seg/data/splitting.py`; use it in the training CLI as `--split specimen`	"sample" means specimen: one physical sample, column `specimen_id` of the manifest	leave `data/raw/` and the manifests unchanged; no new dependencies; [illegible] jobs	each specimen in exactly one fold; each patch [illegible] its specimen; [illegible] the [illegible] folds; [illegible] value if `specimen_id` is missing or there are fewer specimens than folds	`pytest tests/data -q` passes; report changed files, test output and assumptions



✔ what repeats is now in files: `AGENTS.md`, `GLOSSARY.md`, ADRs, `CODING_STANDARDS.md`

The same request, in a prepared repository

the request

```
> Add validation folds by specimen to the training CLI, as decided in ADR-0003.
```

where the rest comes from

Objective	Context	Constraints	Acceptance	Verification
`>_ the request`	`GLOSSARY.md` specimen, fold `ADR-0003` why; how folds are saved	`AGENTS.md` boundaries	`ADR-0003` one fold per specimen `CODING_STANDARDS.md` seed, input checks	`AGENTS.md` fast check, report

still open: what no file says, e.g. the default number of folds

05

Version control with git

How do we keep every version, and see exactly what an agent changed?

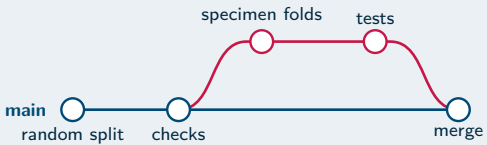
Version control: one folder, every version

Without

```
splitting.py
splitting_old.py
splitting_v2_final.py
splitting_v2_final_FIXED.py
results_new2/
```

Which version produced figure 3?

With git



commit a snapshot with a message, an author and a date
diff what changed between two snapshots
branch a separate line of work, merged when ready
remote a copy on a server (GitHub): backup, sharing, review

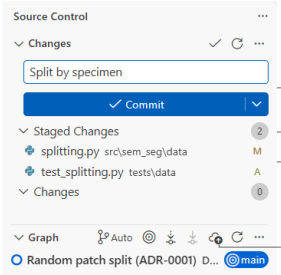
Getting started takes five commands

terminal

```
$ git init              # once: start the history
$ git status            # what has changed?
$ git add src/ tests/   # choose what goes in
$ git commit -m "Split by specimen"
$ git push              # copy it to GitHub
```

.gitignore

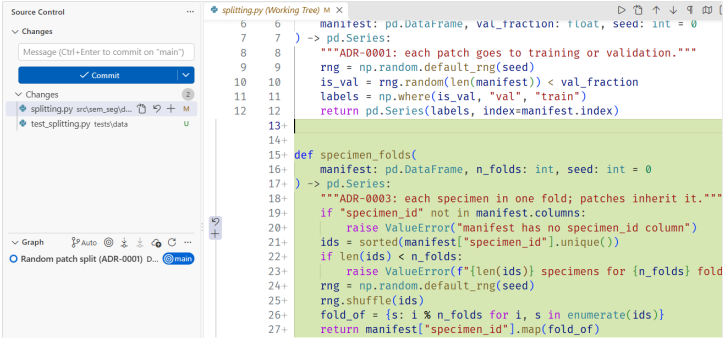
```
data/           # raw and derived data stay out
.venv/          # environments are rebuilt, not stored
*.ckpt          # model checkpoints
.env            # secrets
.claude/worktrees/  # agent worktrees
```



VS Code's Source Control panel does the same with clicks.

data, environments and secrets do not belong in the history

Why an agent needs version control



An agent's change in VS Code's Source Control view: two files, every new line in green.

checkpoint
commit before the agent starts

the agent's edits
read the diff, not the summary

keep `git commit`
undo later: `git revert`

discard `git restore`
the edits are lost

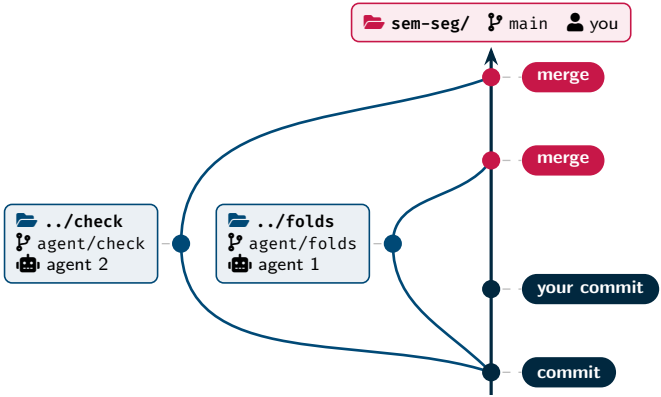
one branch or worktree per task

Worktrees: work in parallel, merge when done

- a worktree: another folder of the same repository, on its own branch
- all folders share one history, so each branch merges into `main`

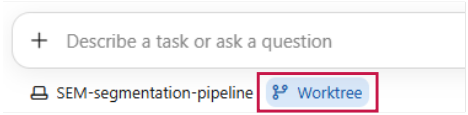
terminal

```
$ git worktree add ../folds \
    -b agent/folds
$ git worktree list
$ git worktree remove ../folds
```



How to use worktrees?

Claude app



Worktree: "Work in an isolated copy of the repository"

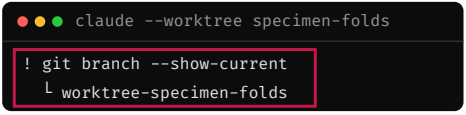
+ New or Ctrl+N, then **Worktree**
off by default, or on per saved environment

`.claude/worktrees/<name>/`
`claude/<name>`

next task: the same again
the sessions run side by side

Create PR, merge, **Archive**

Terminal



the session runs on its own branch

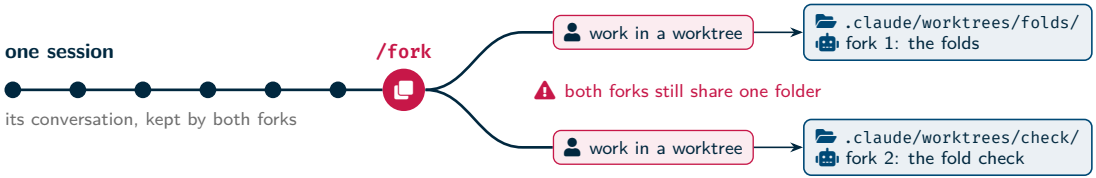
`claude -w specimen-folds`
first time: trust the folder once

`.claude/worktrees/specimen-folds/`
`worktree-specimen-folds`

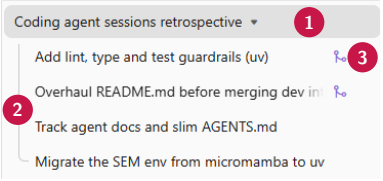
another terminal, another name
another agent

exit

Tip: when a session splits into two tasks, fork it



Claude app



1. one session
2. its forks, one per task
3. pull request merged

Terminal

```
second terminal
$ claude -c --fork-session
> work in a worktree
```

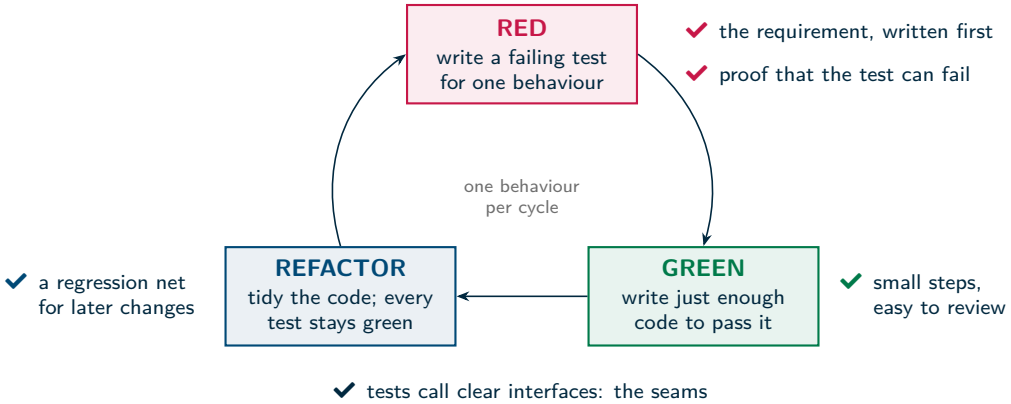
forks the latest session in this folder

06

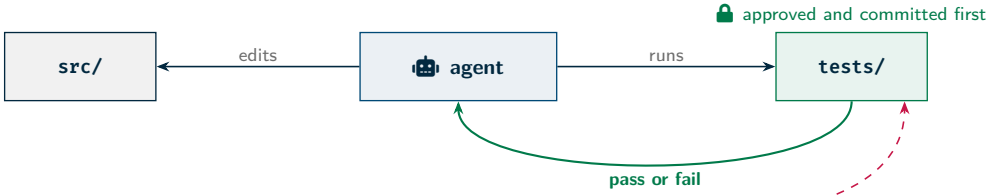
Tests first

How does a test written before the code keep an agent on track?

Test-Driven Development (TDD)



Why tests first help an agent



what it gives the agent

- ✓ a check it runs itself
- ✓ the intent, fixed first
- ✓ one small goal per cycle
- ✓ earlier decisions still enforced

how it gets gamed

- ✗ tests edited or skipped
- ✗ expected values copied from the code
- ✗ a mock that replaces the logic
- ✗ tests fitted after the code

caught by: test diffs reviewed first · you rerun every test



07

Skills and an end-to-end workflow

How does a request become a reviewed change?

A skill: a lab protocol for the agent

In the lab a protocol

Sample preparation

Use when: a specimen goes to the SEM.

1. Grind.
2. Polish.
3. Etch.
4. Check under the light microscope.

written once; everyone in the lab follows it

name
when
steps

In the repository a skill

tdd/SKILL.md shortened

```
name: tdd
description: Test-driven development. Use when
  the user wants to build features or fix
  bugs test-first, ...
1. Agree the seams to test.
2. Write one failing test.
3. Write just enough code to pass.
4. Repeat for the next behaviour.
```

written once; the agent follows it when needed

one procedure per kind of task

worth writing when the task recurs

A skill: procedures loaded when needed

tdd/SKILL.md shortened

```
---
name: tdd
description: Test-driven development. Use when the user
  wants to build features or fix bugs test-first,
  mentions "red-green-refactor", or wants integration
  tests.
---
# Test-Driven Development
## What a good test is
Tests verify behavior through public interfaces, not
implementation details. ...
See [tests.md](tests.md) for examples and
[mocking.md](mocking.md) for mocking guidelines.
```

from the start, like `AGENTS.md`
the name and description of every skill

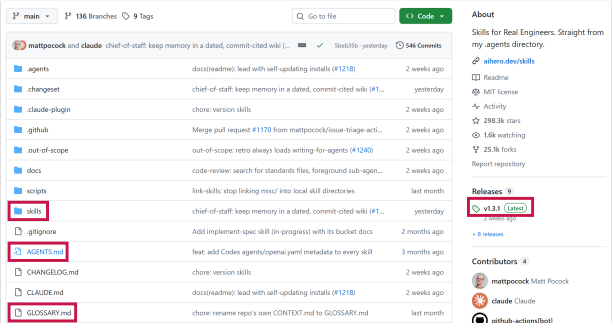
when it runs
`/tdd`, or a task that matches

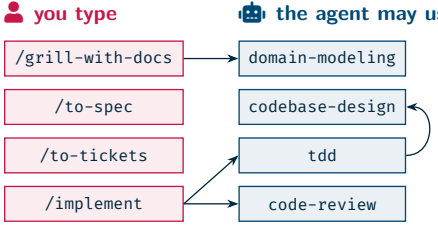
when the steps need them
`tests.md`, `mocking.md` in the same folder

`.claude/skills/` (Claude Code), `.agents/skills/` (Codex)

Matt Pocock's skills: an end-to-end workflow

mattpocock/skills





install

```
$ npx skills@latest add mattpocock/skills
```

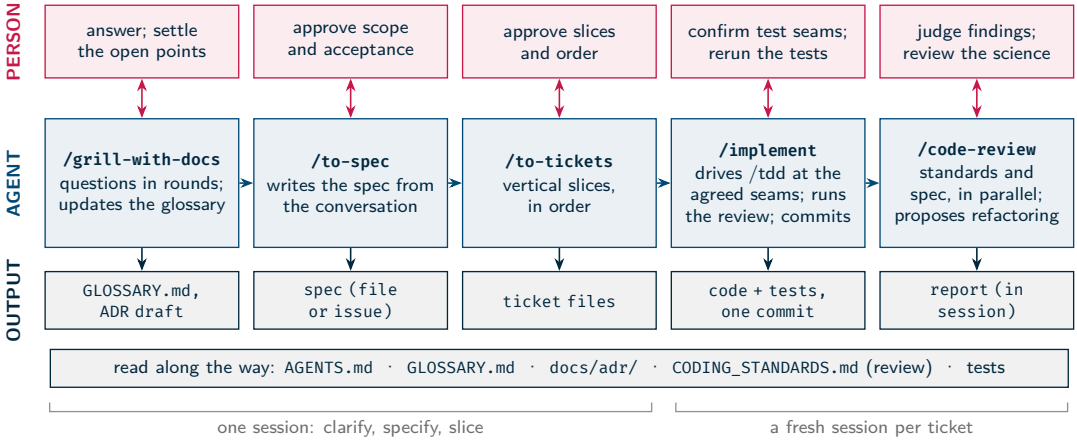
read them first

Classic practice, adapted for agents

The Pragmatic Programmer Andrew Hunt, David Thomas 1999	**Extreme Programming Explained** Kent Beck 1999	**Refactoring** Martin Fowler 1999	**Test-Driven Development: By Example** Kent Beck 2002	**Domain-Driven Design** Eric Evans 2003	**A Philosophy of Software Design** John Ousterhout 2018
no one knows exactly what they want	user stories; invest in the design every day	code smells	red, green, refactor	a ubiquitous language	deep modules
`/grill-with-docs`	`/to-spec`	`/code-review`	`/tdd`	`GLOSSARY.md` `/domain-modeling`	`/codebase-design`

Hunt & Thomas (1999) · Beck (1999; 2002) · Fowler (1999) · Evans (2003) · Ousterhout (2018) · Pocock (2026), mattpocock/skills v1.3.1: README, code-review, codebase-design

Pocock's flow: who does what



A /grill-with-docs session

example session

```
> /grill-with-docs Add validation folds by specimen to the
  training CLI, as decided in ADR-0003.
? Q1 Default number of folds: the manifest lists (1)
  three specimens. One fold per specimen?
→ Yes: k = number of specimens; --folds can lower it. (2)

? Q2 What validation claims: each specimen is the only
  one of its material state, a term GLOSSARY.md lacks.
  Report folds as unseen material states, and add it?
→ Yes.

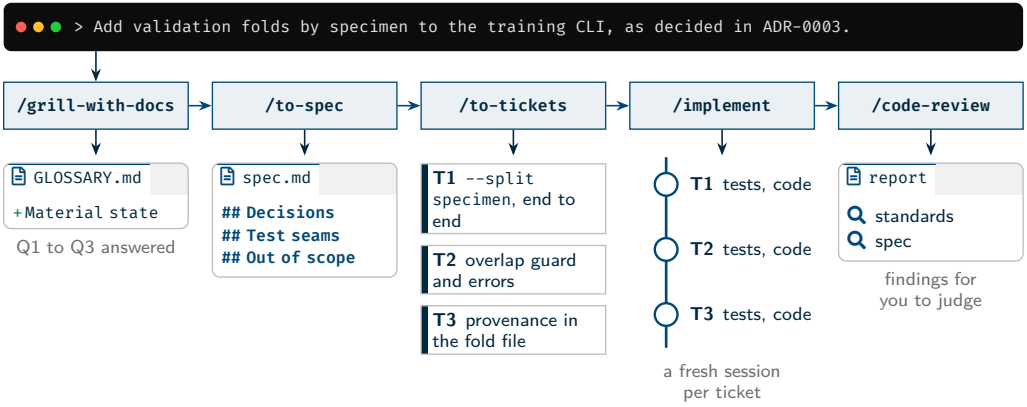
? Q3 Rows without specimen_id: stop with a ValueError,
  as CODING_STANDARDS.md asks?
→ Yes, and name the rows in the message.
> yes to all
Updated GLOSSARY.md: Material state.
No ADR: the number of folds is easy to change later. (3)
```

1. a fact the agent looked up
2. a recommended answer
3. an ADR only if hard to reverse

GLOSSARY.md new entry

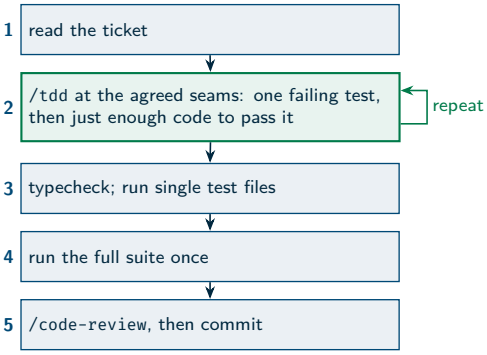
```
Material state: the processing
  condition of a specimen (AsCast,
  Q980_0h_WC, Q980_9h_AC); here,
  one specimen per state.
```

The specimen split, stage by stage



How `/implement` uses `/tdd`

`/implement`: one ticket, one fresh session

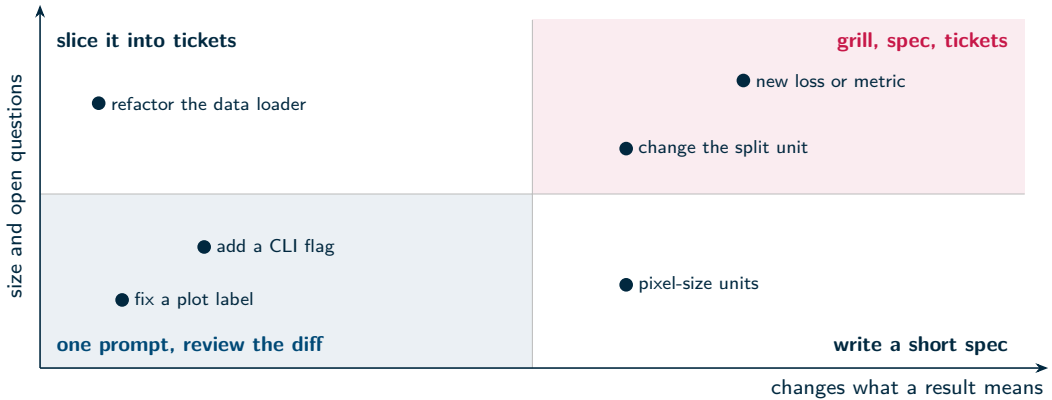


Rules of the `/tdd` loop

- seams agreed with you first
- one failing test, then minimal code
- one vertical slice per cycle
- tests only through public interfaces
- mocks only at system boundaries
- refactoring left to `/code-review`

Known gaps: code written before its test; ticket left open; the run reviews its own work.

When is the full flow worth it?





08

A realistic starting point

What should I change in my repository first?

Other skill packages

`mattpocock/`
`skills`

Matt Pocock

small skills you combine

grill, spec, tickets, implement, review

✓ `/tdd`: red before green, at agreed seams

`obra/`
`superpowers`

Jesse Vincent

a whole method in 15 skills

brainstorm, plan, execute, review, finish

✓ no production code without a failing test first

`multica-ai/`
`andrej-karpathy-skills`

from Karpathy's notes

one `CLAUDE.md`, four rules

think first, keep it simple, change only what is asked, set checkable goals

✓ a bug fix starts with a test that reproduces the bug

✓✓ **in all three, the check is set before the code is written**

Checklist for your repository

Groundwork

- ☐ a git repository; commit before each agent task
- ☐ the Python environment (conda, uv, etc.)
- ☐ Ruff and mypy configured, run after every edit

Context

- ☐ `AGENTS.md`: commands, boundaries, pointers
- ☐ `GLOSSARY.md`: the key terms
- ☐ `CODING_STANDARDS.md`: what review checks
- ☐ `docs/adr/`: decisions people reopen

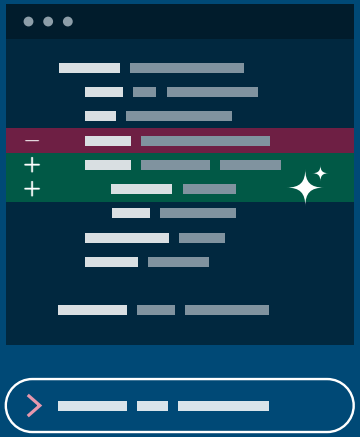
Workflow

- ☐ a workflow that suits you, e.g. `mattpocock/skills`
- ☐ your own skills for the tasks you repeat
- ☐ tests agreed before the code
- ☐ one worktree, or a forked session, per task
- ☐ you review the code and the science

UNIVERSITÄT
DES
SAARLANDES

Thank you

Camilo Martínez



A1 Template: AGENTS.md

AGENTS.md

```
# <project>: notes for coding agents
## Setup and checks
- Environment: `<create and activate command>`
- Fast check (CPU, synthetic data, under a minute): `<command>`
- Lint and types: `<command>`
## Where things are
- Vocabulary: `GLOSSARY.md`; decisions: `docs/adr/`
## Conventions the code does not show
- <units, coordinate frames, file formats, splitting rules>
## Boundaries
- Read-only: `data/raw/`, <other protected paths>
- Ask before: new dependencies, deleting files, cluster jobs
- Never: commit secrets or data; push to `main`
- Never edit a test just to make it pass
## When you finish
- Changed files, commands run, assumptions, open questions
```

- each line prevents a mistake you have seen
- point to documents; do not copy them
- update it in the commit that makes it stale
- Codex: nested files per sub-package; an override replaces one folder's file
- other tools read other names, e.g. `CLAUDE.md`

A2 Templates: glossary, coding standards and ADR

GLOSSARY.md

```
# Glossary
<Term>: one-sentence definition (`code_name`).
  Relates to: <other terms and how>.
  Avoid: <synonyms we do not use>.
<Term>: ...
```

CODING_STANDARDS.md

```
# Coding standards
- Randomness: <how seeds are passed>
- Errors: <what public functions raise>
- Tests: <fixtures, seams, expected values>
- <a judgement call a reviewer can check>
```

docs/adr/NNNN-short-title.md

```
# ADR-NNNN: <short noun phrase>
Status: proposed | accepted |
        superseded by ADR-NNNN
## Context
Forces at play: data, constraints, question.
## Decision
What we will do, in full sentences.
## Considered options
Alternatives and why they lost.
## Consequences
What follows, including the costs.
```

A3 The upgrade: uv

A uv project

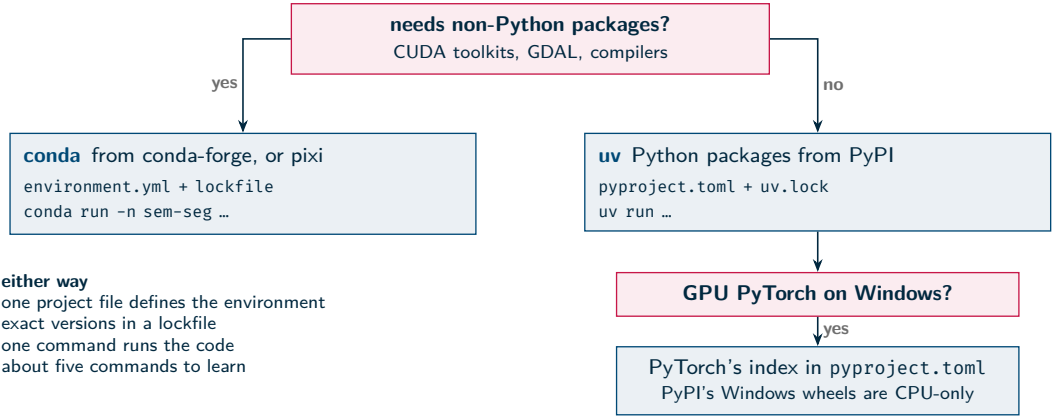
sem-seg/	all committed except .venv/
pyproject.toml	what the project needs
uv.lock	exact versions, every platform
.python-version	the Python version
.venv/	rebuilt from uv.lock

terminal

```
$ uv init sem-seg            # new project
$ uv python pin 3.12         # .python-version
$ uv add numpy pandas torch  # updates uv.lock
$ uv add --dev pytest ruff mypy
$ uv run pytest -q           # sync, then run
$ uv sync                    # rebuild .venv
```

- one tool instead of pip, venv, pip-tools, pipx and pyenv
- much faster than pip
- `uv run`: nothing to activate; the environment is synced first

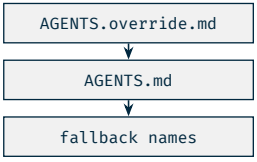
A4 conda or uv?



either way
one project file defines the environment
exact versions in a lockfile
one command runs the code
about five commands to learn

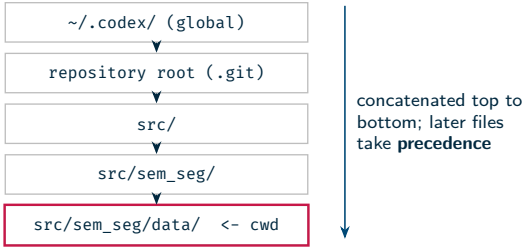
A5 Codex: how instruction files are found

Within one folder: replacement



At most **one file per folder**: an override *replaces* the `AGENTS.md` beside it.

Across folders: layering



Folders below or beside `cwd`: **not** loaded.

Built **once per run**: restart Codex after editing an instruction file.

`project_doc_max_bytes` (default 32 KiB) limits instruction *text*; it is not the model's context window.

Check what loaded: ask Codex to summarize its current instructions.

A6 Pocock's workflow, then and now

	Video (16 July 2026, between v1.1 and v1.2)	Documentation, v1.3.1 (October 2026)
Domain file	`CONTEXT.md` plus ADRs	`GLOSSARY.md` plus ADRs; the skills read only `GLOSSARY.md`
Main flow	`/grill-with-docs`, then `/implement` directly, or `/to-spec`, `/to-tickets`, `/implement` (which runs `/code-review`)	`/grill-with-docs`, `/to-spec`, `/to-tickets`, `/implement` (which drives `/tdd` and runs `/code-review`); `/implement-spec` for a whole spec
Grilling	one question at a time	numbered rounds, each question with a recommended answer
Local tickets	one `tickets.md`	one file per ticket under `.scratch/<feature>/issues/`
Context	"smart zone" of about 140k tokens (heuristic); clear before each ticket	about 150k tokens (heuristic); one session for clarifying to ticketing, a fresh one per ticket
Testing	not shown	`/tdd`: red to green at agreed seams; refactoring moved to `/code-review`
Installation	`npx skills@latest add mattpocock/skills`	Claude Code plugin recommended; `npx` for other agents

A7 References: tools and workflow

OpenAI (2026). *Custom instructions with AGENTS.md*. https://learn.chatgpt.com/docs/agent-configuration/agents-md, accessed 19 Oct 2026.

OpenAI (2026). Codex documentation: *Configuration reference*, *Agent approvals and security*, *Non-interactive mode*, *Build skills*. https://learn.chatgpt.com/docs, accessed 19–22 Oct 2026.

AGENTS.md: a simple, open format for guiding coding agents. https://agents.md/, accessed 19 Oct 2026.

Agent Skills format. https://agentskills.io/, accessed 19 Oct 2026.

Anthropic (2026). Claude Code documentation: *Best practices*, *Memory*, *Checkpointing*, *Tools reference*, *Worktrees*, *Desktop*, *CLI reference*, *Sessions*. https://code.claude.com/docs, accessed 20 Oct 2026, *Sessions* 4 Nov 2026.

Anthropic (2026). *Prompting Claude Opus 5.5*. https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-opus-5-5; OpenAI (2026). *Using GPT-6*. https://developers.openai.com/api/docs/guides/latest-model; both accessed 4 Nov 2026.

M. Pocock (2026). *mattpocock/skills*, v1.3.1, commit 49dd158 (9 Oct 2026), MIT licence. https://github.com/mattpocock/skills

M. Pocock (2026). *AI skills for real engineers*, https://www.aihero.dev/skills, and *v1.3: /implement-spec, /pr, /retro, and GLOSSARY.md*, 5 Oct 2026, https://www.aihero.dev/skills-changelog-v13-implement-spec-pr-retro-and-glossary-md

M. Pocock (2026). *mattpocock/skills: A complete AI coding workflow, end-to-end*. YouTube, 16 Jul 2026. https://www.youtube.com/watch?v=M6mYodf0dJM

A8 References: git, environments and checks

S. Chacon, B. Straub (2014). *Pro Git*, 2nd ed. Apress. https://git-scm.com/book; git documentation, *git-worktree*, https://git-scm.com/docs/git-worktree, accessed 20–21 Oct 2026.

Layout of the worktree diagram, accessed 21 Oct 2026: Anthropic Academy (2026), figure of worktree branches merged into main, https://academy.claude.com/assets/media/536e6bef11940caea62b1d41d10070eb1e2a220811cb0ff388da1af20b4d2c79.png; DevToolLab (2026), worktrees guide, https://devtoollab.com/blog/claude-code-git-worktrees-parallel-agents-guide.

conda (2026). Documentation, v26.9: *Managing environments*, *conda run*, *New features*. https://docs.conda.io, accessed 20 Oct 2026.

Anaconda (2025). *Terms of service*, 15 Jul 2025; *Academic policy*, 14 Jul 2025. https://www.anaconda.com/legal/terms/terms-of-service

conda-forge (2026). *Miniforge*. https://github.com/conda-forge/miniforge; prefix.dev (2026). *pixi documentation*, v0.82. https://pixi.prefix.dev

Astral (2026). *uv documentation*, v0.13: projects, running commands, PyTorch integration. https://docs.astral.sh/uv/, accessed 20 Oct 2026; README with its speed benchmark, https://github.com/astral-sh/uv.

Astral (2026). *Ruff documentation*, v0.17: rules, configuration, editor setup. https://docs.astral.sh/ruff/

mypy documentation, v2.4. https://mypy.readthedocs.io; Microsoft (2026). *Pylance* README and FAQ, https://github.com/microsoft/pylance-release; *Mypy Type Checker*, https://github.com/microsoft/vscode-mypy; VS Code documentation: *Source control*, *Python environments*, *Editing Python*, https://code.visualstudio.com/docs, accessed 20 Oct 2026.

A9 References: methods and evidence

K. Beck (2002). *Test-Driven Development: By Example*. Addison-Wesley.

M. Nygard (2011). *Documenting architecture decisions*. Cognitect blog, 15 Nov 2011.
https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

DrivenData (2026). *Cookiecutter Data Science*, v2: a project layout for data science.
https://cookiecutter-data-science.drivendata.org/, accessed 4 Nov 2026.

N. S. Mathews, M. Nagappan (2024). *Test-driven development for code generation*. ASE '24.
https://doi.org/10.1145/3691620.3695527; arXiv:2402.13521

S. Kapoor, A. Narayanan (2023). *Leakage and the reproducibility crisis in machine-learning-based science*. Patterns 4(9), 100804.
https://doi.org/10.1016/j.patter.2023.100804

B.-I. Bachmann, M. Stiefel, M. Müller, F. T. Mücklich (2026). *A systematically designed open-access SEM dataset and robustness benchmark for machine learning-based microstructure segmentation: a high-chromium cast iron case study*. Zenodo, v1,
CC BY 4.0. https://doi.org/10.5281/zenodo.21931379; DFG project MU 959/57-1.

A10 References: books and skill packages

A. Hunt, D. Thomas (1999). *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley; 20th anniversary ed., D. Thomas, A. Hunt (2019), *The Pragmatic Programmer: Your Journey to Mastery*.

K. Beck (1999). *Extreme Programming Explained: Embrace Change*. Addison-Wesley; 2nd ed. with C. Andres, 2004.

M. Fowler (1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley; 2nd ed. 2018. Chapter 3: bad smells in code.

E. Evans (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

J. Ousterhout (2018). *A Philosophy of Software Design*. Yaknyam Press; 2nd ed. 2021.

Agent Skills (2026). *Specification*. `https://agentskills.io/specification`; Anthropic (2026). Claude Code documentation: *Skills*. `https://code.claude.com/docs/en/skills`, both accessed 3 Nov 2026.

J. Vincent, Prime Radiant (2026). *obra/superpowers*, v7.0.1 (21 Oct 2026), MIT licence. `https://github.com/obra/superpowers`

J. Zhang (2026). *andrej-karpathy-skills*, a `CLAUDE.md` derived from A. Karpathy's observations on LLM coding pitfalls. `https://github.com/multica-ai/andrej-karpathy-skills`, accessed 3 Nov 2026.